

SIMULATION & RESEARCH PROTOTYPE

Intelligent RF Spectrum Scan Strategy

Product Requirements & Technical Design Specification

AI/ML-Driven Scan Scheduler for a Simulated Electronic Warfare Environment

Document Version: 1.0 | Status: Draft for Engineering Review

Classification: Unclassified — Simulation-Only Prototype

Prepared by: Product & Engineering Organization (PM, Architecture, AI/ML, Backend, Frontend, QA, DevOps, Security)

Date: August 2026

SCOPE: SOFTWARE SIMULATION ONLY — NO REAL RF HARDWARE, INTERCEPTION, JAMMING OR WEAPON CONTROL

Table of Contents

1. Executive Summary	15. WebSocket Specification
2. Problem Statement & Background	16. Security
3. Goals & Non-Goals	17. Testing Strategy
4. Stakeholders & Personas	18. DevOps & Deployment
5. Functional Requirements	19. Observability & Logging
6. Non-Functional Requirements	20. UX/UI & Demo Mode
7. System Scope: MVP / V1 / V2	21. Project Directory Structure
8. Simulation Engine Design	22. Development Plan & Phases
9. Receiver Model	23. Team Ownership
10. AI/ML Scheduler & RL Formulation	24. Risks & Mitigations
11. Periodic Emitter Handling	25. Assumptions & Constraints
12. Metrics & Evaluation	26. Technology Decision Records
13. Experiment Design	27. Requirement Traceability Matrix
14. High-Level & Low-Level Design	28. Definition of Done & Appendix

Document length target: ≤ 40 pages. Diagrams are rendered as vector graphics (SVG), not as diagram-source code blocks.

1. Executive Summary

This document specifies the product requirements and technical design for a **software-based, simulation-only research prototype** that studies intelligent RF spectrum scan scheduling. A receiver with limited instantaneous bandwidth cannot observe an entire simulated spectrum at once. A fixed, open-loop scan pattern wastes time revisiting quiet bands and can miss or delay detection of periodic or frequency-agile emitters. This system builds a configurable RF environment simulator, a receiver model, an open-loop baseline scanner, and a machine-learning-driven scheduler that learns — from simulated scan outcomes — which band to observe next, and for how long, in order to maximize detection while minimizing wasted scan time.

The prototype is intentionally scoped as a **research and demonstration platform**. It does not control real RF hardware, does not intercept real-world communications, and does not perform jamming, electronic attack, or weapon targeting of any kind. All emitters, signals, and detections exist only inside a synthetic, deterministic-by-seed simulation.

The recommended MVP scheduler is a **contextual multi-armed bandit**, upgradeable to tabular Q-Learning and, as a stretch goal, a Deep Q-Network (DQN), based on the trade-off analysis in Section 26. The system is evaluated against the open-loop baseline using probability of detection (Pd), probability of false alarm (Pfa), average intercept time, and detection latency, across seven emitter-mix scenarios (Section 13). A live dashboard visualizes the spectrum, receiver state, scheduler decisions, and baseline-vs-ML comparison in a scripted 3-5 minute demo (Section 20).

1.1 Key Outcomes

Outcome	Description
Simulated RF environment	Configurable spectrum with 5 emitter behavior classes, deterministic ground truth per band/time-slot.
Baseline scanner	Round-robin, fixed-interval, configurable dwell-time open-loop scanner used as the control group.
ML scheduler	Learns a scan policy from reward signal; MVP = contextual bandit; optional Q-Learning/DQN escalation.
Evaluation harness	Reproducible experiments across 7 scenarios, with Pd, Pfa, intercept time, and scan efficiency reported.
Dashboard	Real-time spectrum, scheduler-decision, and metrics visualization for technical and non-technical viewers.

2. Problem Statement & Background

2.1 Problem Statement

A receiver has a limited instantaneous bandwidth and cannot observe the entire spectrum simultaneously. Traditional open-loop scanning follows a fixed pattern (e.g., round-robin across bands at a fixed dwell time), which can cause important emitters to be missed or detected late — particularly when emitters are periodic, frequency-agile, or otherwise dynamic. The system must produce a simulated RF environment and an AI/ML-driven scan scheduler that learns, from previous scan outcomes, which band to scan, when, and for how long, in order to maximize useful detection while minimizing unnecessary scanning and intercept time.

2.2 Background

Open-loop, round-robin scanning is simple and predictable but treats every band as equally likely to carry activity at every point in time. In real spectra, activity is highly non-uniform: some emitters transmit continuously, some are periodic bursts, and some hop frequency pseudo-randomly. A scheduler that models band activity from its own scan history can allocate more of the receiver's limited dwell time to bands that are currently or predictably active, and less to bands that are historically quiet — a classic explore/exploit resource-allocation problem, which maps naturally onto multi-armed bandit and reinforcement-learning formulations.

2.3 Why This Matters for the Prototype

- Demonstrates a measurable, reproducible improvement of a learned policy over a fixed baseline on identical simulated scenarios.
- Provides an extensible research platform: new emitter behaviors, receiver constraints, and scheduling algorithms can be added without re-architecting the system.
- Keeps the entire capability inside a closed simulation boundary appropriate for a hackathon/research prototype (see Section 16, Security).

3. Goals & Non-Goals

3.1 Goals

1. Build a configurable, reproducible simulated RF spectrum with ground-truth labels.
2. Model a bandwidth-constrained receiver with realistic scan/tune/dwell constraints.
3. Implement an open-loop baseline scanner as the control condition.
4. Implement a learning-based scheduler (bandit → Q-Learning → optional DQN/PPO) that outperforms the baseline on defined metrics.
5. Provide a mathematically defined, reproducible evaluation harness (Pd, Pfa, intercept time, latency, efficiency).
6. Explicitly model and evaluate periodic-emitter prediction.
7. Provide a real-time dashboard suitable for technical and non-technical audiences.
8. Keep the system strictly simulation-only, auditable, and extensible.

3.2 Non-Goals

- No control of, or interface to, real RF hardware or SDR devices.

- No real-world signal interception, direction-finding, or geolocation of actual emitters.
- No jamming, spoofing, electronic-attack, or countermeasure functionality.
- No weapon-system targeting, cueing, or fire-control integration.
- No use of classified, export-controlled, or operationally sensitive data or waveforms.

4. Stakeholders & Personas

Persona	Description	Primary Needs
Research Engineer	Builds and tunes the simulator, receiver model, and ML scheduler.	Configurable scenarios, reproducibility, fast iteration, metrics logging.
Evaluator / Judge	Non-technical or semi-technical reviewer assessing the prototype.	Clear visual comparison of baseline vs. ML, quick 3-5 minute demo.
ML Practitioner	Trains, evaluates, and versions scheduling models.	Clean state/action/reward API, checkpointing, experiment tracking.
Backend/Frontend Engineer	Implements APIs, persistence, and dashboard.	Well-defined contracts, schemas, and WebSocket events.
QA / Security Reviewer	Validates functional correctness and simulation-only boundary.	Test matrix, traceability, explicit scope boundary documentation.

5. Functional Requirements

ID	Requirement
FR-001	The system shall allow creation of a simulation with a configurable number of frequency bands, duration, and random seed.
FR-002	The system shall support at least 5 emitter behavior classes: fixed-frequency, periodic, frequency-agile, random/pseudo-random, and intermittent.
FR-003	The simulator shall generate a per-band, per-time-slot ground-truth activity label independent of any scan decision.
FR-004	The receiver model shall expose a configurable instantaneous bandwidth (number of bands observable per scan), dwell time, and tuning/switching delay.
FR-005	The system shall implement an open-loop baseline scanner supporting round-robin and configurable fixed-order scanning.
FR-006	The system shall implement a learned scheduler that selects the next band(s) to scan based on scan/detection history.
FR-007	The scheduler shall support pluggable algorithms: multi-armed bandit (MVP), Q-Learning (V1), DQN/PPO (V2).
FR-008	The system shall compute Pd, Pfa, average intercept time, detection latency, interception ratio, and scan efficiency for any completed simulation run.
FR-009	The system shall support an Experiment entity that runs the same scenario under baseline and ML policies for direct comparison.
FR-010	The system shall persist scan events, detection events, and scheduler decisions for later analysis.
FR-011	The system shall expose a model registry supporting training, versioning, activation, and evaluation of scheduler models.
FR-012	The system shall stream live simulation state (current scan, detections, metrics) to the frontend via WebSocket.
FR-013	The dashboard shall visualize spectrum activity, current receiver focus, scheduler decisions, and baseline-vs-ML comparison charts.
FR-014	The system shall provide a scripted demo mode executing a full baseline-vs-ML comparison in 3-5 minutes.
FR-015	The system shall estimate periodicity of previously detected emitters and expose the predicted next-active-window to the scheduler.

6. Non-Functional Requirements

ID	Category	Requirement
NFR-001	Performance	API p95 response latency < 200 ms for CRUD endpoints under nominal load.
NFR-002	Performance	Scheduler decision latency < 50 ms per step for bandit/Q-Learning; < 150 ms for DQN inference.
NFR-003	Performance	WebSocket event delivery latency < 250 ms end-to-end under demo load ($\leq$ 10 concurrent viewers).
NFR-004	Scalability	System shall support $\geq$ 5 concurrent simulations, $\geq$ 64 bands, and $\geq$ 100 emitters per simulation.
NFR-005	Reliability	Simulation state shall be checkpointed at least every 500 simulation steps to allow resume after failure.
NFR-006	Reproducibility	Given identical seed and configuration, simulator output shall be bit-for-bit reproducible.
NFR-007	Maintainability	Backend and ML modules shall maintain $\geq$ 80% unit-test coverage on core logic (simulation, reward, metrics).
NFR-008	Observability	Every simulation, experiment, and training run shall carry a correlation ID present in all related logs.
NFR-009	Usability	A first-time, non-technical viewer shall be able to interpret the baseline-vs-ML comparison chart without narration.
NFR-010	Security	The system shall have no code path capable of addressing physical RF hardware or external network radios.

7. System Scope: MVP / V1 / V2

Tier	Included Capabilities
MVP	Simulated spectrum with all 5 emitter classes; receiver model; open-loop baseline; contextual multi-armed bandit scheduler;

(hackathon deliverable)	Pd/Pfa/intercept-time/latency metrics; single-scenario experiment run; live dashboard with spectrum view, scheduler view, and baseline-vs-ML comparison chart; 3-5 minute scripted demo; REST + WebSocket API; PostgreSQL persistence; Dockerized deployment.
V1 (post-hackathon)	Tabular Q-Learning scheduler as selectable alternative; full 7-scenario experiment suite; periodicity estimator with predicted next-active-window feeding scheduler state; model registry with versioning/rollback; authentication; experiment comparison reports (exportable).
V2 (advanced research)	Deep RL (DQN/PPO) scheduler; contextual bandit with neural function approximation; multi-receiver / distributed sensing scenarios; transfer learning across scenario families; automated hyperparameter search; adversarial/frequency-agile stress scenarios with online model adaptation.

MVP scope deliberately excludes Deep RL: a contextual bandit is the simplest algorithm capable of demonstrating measurable improvement over the open-loop baseline within hackathon time and compute constraints (see Section 26 for the full trade-off analysis).

8. Simulation Engine Design

8.1 Core Entities

Entity	Responsibility
SimulationClock	Advances discrete simulation time in fixed-size steps; drives all other components.
Spectrum	Owns the set of FrequencyBand objects and their ground-truth state over time.
FrequencyBand	Represents one scannable band: center frequency, width, and current occupancy.
Emitter	Generates activity in one or more bands according to an EmitterBehavior policy.
EmitterBehavior	Strategy object implementing fixed / periodic / agile / random / intermittent activity generation.
Signal	An instantaneous activity event produced by an Emitter in a band/time-slot.
Receiver	Holds instantaneous bandwidth, dwell time, tuning delay, and detection threshold.
Scanner	Executes a scan action against the Receiver and Spectrum, producing an Observation.
DetectionEngine	Applies the detection threshold/noise model to an Observation to emit Detection/Miss/FalseAlarm events.
GroundTruthGenerator	Pre-computes/streams the ideal activity table used only for metric scoring, never given to the scheduler.

8.2 Simulation Loop (Pseudocode)

```
for t in range(0, T, step):
    spectrum.advance(t)                # update ground-truth occupancy
    action = scheduler.decide(state)    # baseline or ML policy
    observation = scanner.execute(action, receiver, spectrum)
    event = detection_engine.evaluate(observation, threshold)
    reward = reward_fn(event, action, state)
    state = state_builder.update(state, action, observation, event)
    metrics.record(t, action, observation, event, reward)
    scheduler.learn(state, action, reward) # no-op for baseline
```

8.3 Emitter Behavior Classes

Class	Definition
Fixed-frequency	Continuously or near-continuously active on one assigned band.
Periodic	Active for a dwell window every T_{period} time units (constant or jittered period).
Frequency-agile	Hops across a defined set of bands according to a hop sequence or hop rate.
Random / pseudo-random	Activates with a stochastic per-slot probability, independent across slots.
Intermittent	Bursty ON/OFF activity with variable-length silent gaps between bursts.

9. Receiver Model

9.1 Formal Definitions

Concept	Definition
Receiver state $\mathbf{R}(t)$	Tuple (bands currently tuned, remaining dwell time, tuning-delay countdown, scan history window).
Scan action $\mathbf{a}(t)$	Subset of bands of size $\leq K$ (instantaneous bandwidth), plus optional dwell time d .
Observation $\mathbf{o}(t)$	Noisy measurement per scanned band: signal-present likelihood or SNR estimate.
Detection event	Observation exceeds the detection threshold and ground truth = active (true positive).
Miss event	Ground truth = active but band not scanned, or scanned observation below threshold (false negative).
False alarm event	Observation exceeds threshold but ground truth = inactive (false positive).
Time progression	$t \rightarrow t + \text{step}$; tuning delay must fully elapse before a newly tuned band yields a valid observation.

9.2 Configurable Parameters

- **Total spectrum:** N configurable bands (default 16, tested up to 64).
- **Instantaneous bandwidth:** K bands observable per step (default $K=2$, $K \leq N$).
- **Dwell time:** minimum observation duration per band, configurable per scan action.
- **Tuning/switching delay:** fixed cost applied when the receiver retunes to a previously unobserved band.
- **Detection threshold:** SNR or likelihood cut-off; tunable to trade P_d against P_{fa} .

The receiver never interacts with physical hardware; all observations are generated by the simulator's noise model applied to the ground-truth occupancy table.

10. AI/ML Scheduler & RL Formulation

10.1 Algorithm Options Considered

Algorithm	Complexity	Training Time	Interpretability	Recommendation
Multi-Armed Bandit (contextual)	Low	Minutes	High	MVP — chosen
Tabular Q-Learning	Medium	Tens of minutes	Medium	V1 — optional escalation
Deep Q-Network (DQN)	High	Hours, GPU preferred	Low	V2 — stretch goal only
PPO	High	Hours, GPU preferred	Low	V2 — research extension

10.2 State, Action, and Reward (ML-001 - ML-003)

ID	Element	Definition
ML-001	State $s(t)$	Vector per band: time since last scan, exponentially-weighted recent detection rate, count of consecutive misses, estimated periodicity phase (Section 11), band priority weight, and receiver tuning cost to reach that band.
ML-002	Action $a(t)$	<code>next_band</code> for the MVP bandit/Q-Learning agent; <code>(next_band, dwell_time)</code> for the V2 DQN/PPO agent once dwell-time control is justified.
ML-003	Reward $r(t)$	See Equation 10.1 below.

10.3 Reward Function

Equation 10.1

$$r(t) = w1 \cdot D(t) + w2 \cdot P(t) \cdot D(t) - w3 \cdot L(t) - w4 \cdot F(t) - w5 \cdot C(t) - w6 \cdot M(t)$$

Term	Meaning
$D(t)$	1 if a true detection occurred on the scanned band at t, else 0.
$P(t)$	Priority multiplier of the detected emitter's threat/priority category (≥ 1).
$L(t)$	Normalized detection latency penalty since the band last became active.
$F(t)$	1 if a false alarm occurred, else 0.
$C(t)$	Redundant-scan penalty: cost of rescanning a band with no new information.
$M(t)$	Missed-opportunity penalty: ground truth active in an unscanned, high-priority band.
$w1...w6$	Non-negative tunable weights (config-driven, logged per experiment).

10.4 Episode, Exploration & Training Definitions

- **Episode:** one full simulation run of fixed duration T; terminal state at $t = T$.
- **Observation space:** per-band feature vectors described in ML-001, flattened for the bandit/tabular agent.
- **Exploration:** ϵ -greedy for bandit/Q-Learning (ϵ decayed per episode); entropy-regularized policy for PPO.
- **Exploitation:** argmax expected reward (bandit/Q-Learning); policy network sampling (DQN/PPO).
- **Checkpointing:** model weights/Q-table snapshot every N episodes and on best-validation-reward.
- **Versioning:** every trained model registered with algorithm, hyperparameters, training-data seed range, and evaluation metrics (see Section 24, ML pipeline).

11. Periodic Emitter Handling

Periodic emitters are the primary motivation for moving beyond open-loop scanning: a fixed round-robin pattern that happens to poll a band during its OFF phase will systematically miss or delay detection. The prototype addresses this with a dedicated, lightweight statistical predictor rather than folding period estimation into the RL policy itself.

11.1 Recommended Architecture: Hybrid Predictor + Scheduler

A separate `PeriodicityEstimator` module tracks each band's detection timestamps and fits a simple autocorrelation/inter-arrival estimate of period length and phase. Its output — *predicted next-active-window* and a confidence score — is exposed as additional **features in the RL state vector** (ML-001), not as a standalone decision-maker. This keeps the RL policy responsible for the actual scan trade-off while the statistical estimator specializes in the narrower, more data-efficient problem of periodicity detection.

Option	Why Not Chosen as Primary
Fold periodicity purely into RL state, no separate model	RL requires far more samples to learn periodicity implicitly than a closed-form estimator; slower convergence for MVP timeline.

Separate ML model (learned periodicity classifier)	Overkill for MVP; a statistical inter-arrival estimator is simpler, interpretable, and sufficient for the defined emitter classes.
Hybrid (statistical estimator feeding RL state) — chosen	Best interpretability/data-efficiency trade-off; isolates periodicity estimation for independent unit testing.

12. Metrics & Evaluation

Metric	Class	Formula
Probability of Detection (Pd)	Primary	$Pd = TP / (TP + FN)$
Probability of False Alarm (Pfa)	Primary	$Pfa = FP / (FP + TN)$
Average Intercept Time	Primary	$AIT = (1/n) \sum (t_{detect,i} - t_{active_start,i})$
Detection Latency	Primary	$Latency_i = t_{detect,i} - t_{active_start,i}$ (per event)
Interception Ratio	Secondary	$IR = (\# \text{ unique emitters detected}) / (\# \text{ unique emitters present})$
Scan Efficiency	Secondary	$SE = (\text{useful scans}) / (\text{total scans})$, useful = scan on an active band
Cumulative Reward	Secondary	$R_{total} = \sum r(t)$ over the episode (Eq. 10.1)
High-Priority Detection Rate	Primary	$HPDR = TP_{highpriority} / (TP+FN)_{highpriority}$
Precision	Secondary	$Precision = TP / (TP + FP)$
Recall	Secondary	$Recall = TP / (TP + FN)$ (equivalent to Pd)
F1	Secondary	$F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall)$
Coverage	Secondary	$Coverage = (\# \text{ distinct bands scanned}) / N$ over episode
Miss Rate	Secondary	$MissRate = FN / (TP + FN) = 1 - Pd$

Primary metrics (Pd, Pfa, average intercept time, detection latency, high-priority detection rate) drive the headline baseline-vs-ML comparison. **Secondary metrics** support deeper diagnostic analysis and hyperparameter tuning.

13. Experiment Design

Every experiment runs the identical scenario configuration under the open-loop baseline and each enabled ML policy, with the same random seed for spectrum generation to ensure a fair comparison.

Scenario	Emitter Mix	Bands	Emitters	Duration
A — Mostly Fixed	80% fixed, 20% mixed	16	10	2,000 steps
B — Mostly Periodic	70% periodic, 30% mixed	16	10	2,000 steps
C — Frequency-Agile	70% agile, 30% mixed	24	12	2,000 steps
D — Mixed Environment	Even split across 5 classes	24	15	3,000 steps
E — High-Density	Mixed, high emitter count	32	30	3,000 steps
F — Sparse	Mixed, low emitter count	32	5	2,000 steps
G — Rapidly Changing	Mixed with dynamic behavior-class switching	24	15	3,000 steps

Every scenario specifies: number of bands, number of emitters, simulation duration, episode count (≥ 20 per policy for statistical stability), random seed set, emitter class distribution, receiver bandwidth (K), dwell time, target metrics, and expected qualitative outcome (documented in the Experiment entity's `expected_outcome` field). Results across baseline and ML runs are stored under a shared `experiment_id` for direct comparison (see API-024-028).

14. High-Level & Low-Level Design

14.1 System Architecture (HLD)

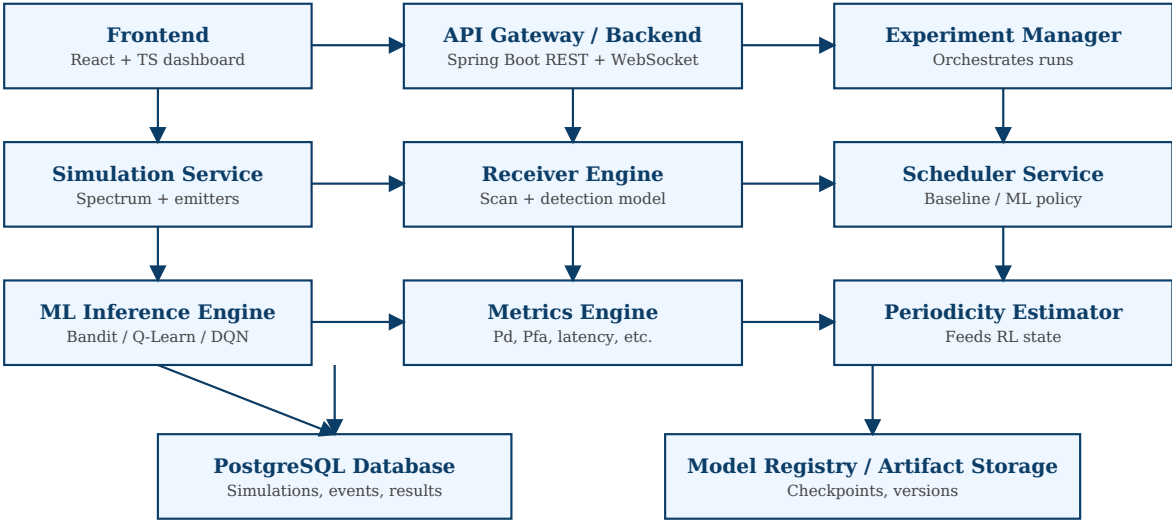


Figure 14.1 — Layered system architecture (Frontend → API → domain services → ML → persistence).

14.2 Data Flow & Deployment Overview



docker-compose orchestrates all four containers on a single host for demo/dev; each service scales independently in V1.

Figure 14.2 — Deployment / data-flow view (single-host docker-compose topology).

14.3 Sequence: Simulation Start → Scan → Detection → Scheduler Decision

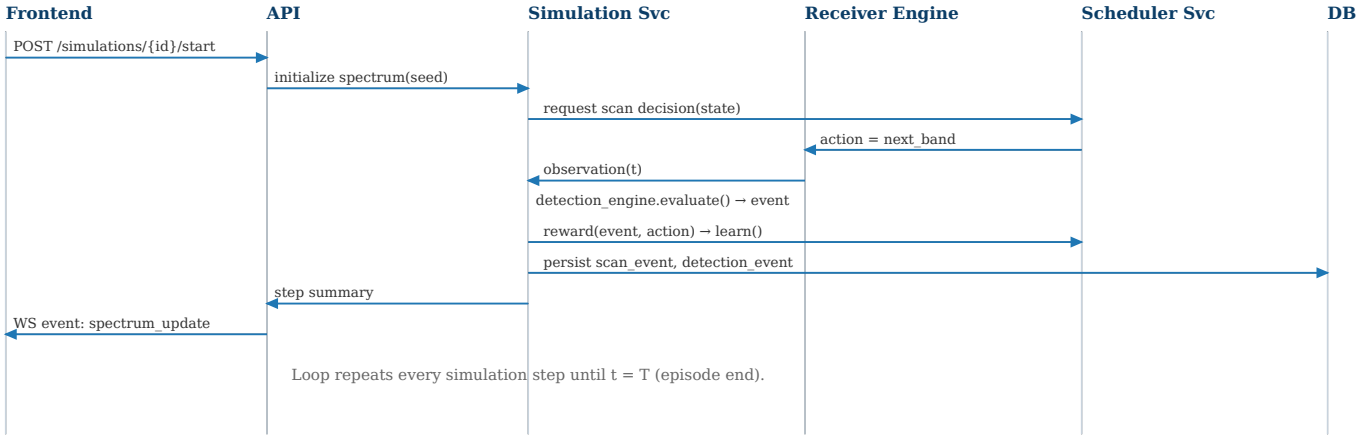


Figure 14.3 — End-to-end scan/detection/learning sequence per simulation step.

14.4 Entity Relationship Diagram

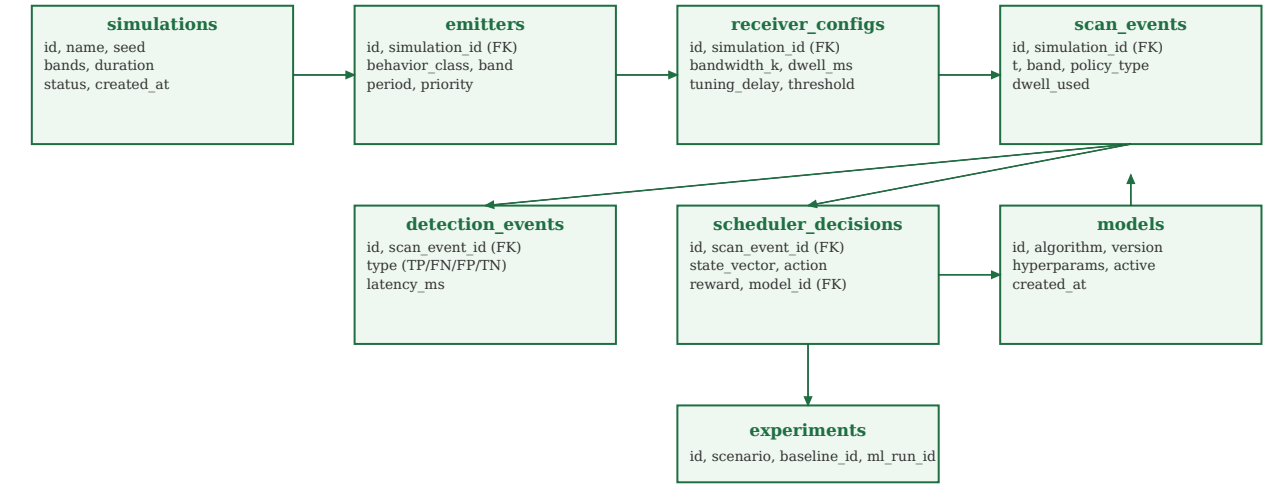


Figure 14.4 — Core entity relationships (simulation → emitters/receiver → scan/detection/decision → experiment).

14.5 Additional LLD Sequence Diagrams (Summary)

Sequence	Key Steps
Model Training	Experiment Manager → ML Inference Engine: generate episodes → Trainer.fit() → Evaluator.score() → Model Registry.save(version).
Experiment Execution	API → Experiment Manager: run baseline episodes → run ML episodes (same seeds) → Metrics Engine.compare() → persist experiment_results.
Dashboard Real-Time Update	Simulation Service emits step event → API WebSocket hub → Frontend store update → chart re-render (≤ 250 ms budget, NFR-003).

15. Backend Architecture, WebSocket & API Specification

15.1 Backend Stack & Layering

The core application backend is built on **Java 21 + Spring Boot 3** (Spring Web MVC, Spring Data JPA, Spring WebSocket, Spring Validation, Spring Security for V1 auth), with **PostgreSQL** as the system of record and **Flyway** for versioned schema migrations. Redis (Spring Data Redis) provides pub/sub fan-out for WebSocket broadcast across instances and backs the async job queue for long-running training/experiment jobs. Layers: `Controller` → `Service` → `Repository` → `Entity`, with request/response DTOs (Java records) validated via `jakarta.validation` annotations, a centralized `@RestControllerAdvice` exception handler mapping domain exceptions to the standard error envelope (Section 20.2), and structured JSON logging via SLF4J + Logback.

The RL/bandit algorithms themselves (Section 15.5) run in a small, dedicated **Python ML microservice** rather than inside the Java backend, since the RL/scientific-computing ecosystem (Gymnasium, NumPy, Stable-Baselines3) is Python-native. The Spring Boot `SchedulerService` calls this microservice over an internal REST contract (`/internal/decide`, `/internal/train`) — this boundary is documented as ADR-06 in Section 26.

15.2 Backend Folder Structure

```
backend/src/main/java/com/rfscheduler/
  controller/      # @RestController per resource (Simulation, Emitter, Receiver, Scheduler, Model, Experiment, Metrics)
  config/          # Spring config: WebSocketConfig, SecurityConfig, RedisConfig, OpenApiConfig
  domain/          # JPA @Entity classes (Simulation, Emitter, ReceiverConfig, ScanEvent, DetectionEvent, ...)
  dto/             # request/response Java records + MapStruct mappers
  service/         # business logic (SimulationService, SchedulerService, MetricsService, ExperimentService)
  repository/      # Spring Data JPA repositories per entity
  simulation/      # SimulationClock, Spectrum, Emitter, EmitterBehavior
  receiver/        # Receiver, Scanner, DetectionEngine
  scheduler/       # BaselineScheduler + MLSchedulerClient (calls the Python ML microservice)
  metrics/         # Pd/Pfa/latency/efficiency calculators
  experiments/     # ExperimentRunner, comparison logic
  websocket/       # STOMP/native WebSocketHandler, session + topic management
  exception/       # domain exceptions + @RestControllerAdvice
RfSchedulerApplication.java

backend/src/main/resources/ # application.yml, db/migration/ (Flyway SQL)
backend/src/test/java/     # unit + integration (Testcontainers + PostgreSQL)
backend/pom.xml (or build.gradle.kts)
```

15.3 REST API — Consolidated Contract

Method	Endpoint	Purpose	Notes
POST	/api/v1/simulations	Create simulation	Body: bands, duration, seed → 201 + simulation_id
GET	/api/v1/simulations	List simulations	Paginated, filterable by status
GET	/api/v1/simulations/{id}	Get simulation detail	404 if not found
PUT	/api/v1/simulations/{id}	Update configuration	Only while status=draft

DELETE	/api/v1/simulations/{id}	Delete simulation	Cascades scan/detection events
POST	/api/v1/simulations/{id}/start	Start execution	Enqueues simulation worker job
POST	/api/v1/simulations/{id}/stop	Stop execution	Persists partial state
POST	/api/v1/simulations/{id}/reset	Reset to t=0	Clears scan/detection history
POST	/api/v1/emitters	Create emitter	behavior_class enum-validated
GET / PUT / DELETE	/api/v1/emitters/{id}	Read / update / delete emitter	Standard CRUD, 404 on missing id
GET	/api/v1/receiver/status	Current receiver state	Includes tuned bands, dwell remaining
PUT	/api/v1/receiver/config	Update receiver params	bandwidth_k, dwell, threshold
POST	/api/v1/receiver/scan	Manual single-step scan (debug)	Returns Observation
GET	/api/v1/scheduler/status	Current policy + step count	
PUT	/api/v1/scheduler/config	Select policy (baseline/bandit/q_learning)	Validated against enabled algorithms
POST	/api/v1/scheduler/start / stop	Start/stop scheduling loop	
GET	/api/v1/scheduler/decision	Latest decision + state vector	Debug/inspection
GET	/api/v1/scheduler/history	Paginated decision log	
POST	/api/v1/models/train	Launch training job	Async; returns job_id
GET	/api/v1/models	List models	Filter by algorithm/active
GET	/api/v1/models/{id}	Model detail + metrics	
POST	/api/v1/models/{id}/activate	Promote to active scheduler model	Deactivates previous active model
POST	/api/v1/models/{id}/evaluate	Run evaluation episodes	Returns metrics summary
POST	/api/v1/experiments	Define experiment (scenario + policies)	
GET	/api/v1/experiments	List experiments	
GET	/api/v1/experiments/{id}	Experiment detail	
POST	/api/v1/experiments/{id}/run	Execute baseline + ML runs	Async; WS progress events
POST	/api/v1/experiments/{id}/stop	Cancel a running experiment	
GET	/api/v1/experiments/{id}/results	Comparison results	Pd/Pfa/latency per policy
GET	/api/v1/metrics/live	Live metrics for a running simulation	Polling fallback for WS
GET	/api/v1/metrics/{experimentId}	Stored metrics for an experiment	
GET	/api/v1/metrics/compare	Compare metrics across ≥2 experiments	Query param: ids[]
GET	/health /ready /metrics	Health, readiness, Prometheus metrics	No auth required

Common conventions: all endpoints return the standard envelope `{success, data|error, requestId}`; validation errors return HTTP 422 with field-level detail; not-found returns 404 with error code `RESOURCE_NOT_FOUND`; authentication (V1) uses bearer JWT on all routes except `/health`, `/ready`.

Example — Create Simulation

Request POST /api/v1/simulations { "name": "Scenario D", "bands": 24, "duration_steps": 3000, "seed": 42 }	Response 201 {"success": true, "data": { "id": "sim_8f2a", "status": "draft", "requestId": "req_019a" } }
---	---

15.4 WebSocket Specification

Property	Definition
Endpoint	/ws/v1/simulations/{simulationId}
Connection lifecycle	Client connects post-simulation-creation; server sends <code>connection_ack</code> ; client may send <code>subscribe</code> for specific channels (spectrum, scheduler, metrics, training).
Event types	<code>spectrum_update</code> , <code>scan_decision</code> , <code>detection_event</code> , <code>metrics_update</code> , <code>training_progress</code> , <code>error</code>
Heartbeat	Server ping every 15 s; client must pong within 10 s or connection is closed.
Reconnection	Client exponential backoff (1s→30s cap); server replays last known state snapshot on reconnect.
Backpressure	Server coalesces <code>spectrum_update</code> events to at most 10/s per connection under load; drops intermediate frames, never the latest.
Error event	{ "type": "error", "code": "SIM_NOT_RUNNING", "message": "... " }

15.5 ML Folder Structure

ml/ environments/ agents/	environment.py, state.py, action_space.py, reward.py # Gymnasium-compatible env bandit_agent.py, q_learning_agent.py, dqn_agent.py, ppo_agent.py
---------------------------------	---

algorithms/	shared exploration/exploitation strategies
features/	periodicity_estimator.py, band_feature_builder.py
training/	trainer.py, hyperparameter configs
evaluation/	evaluator.py, metrics comparison helpers
inference/	inference.py – low-latency decision API used by SchedulerService
checkpoints/	versioned weights / Q-tables
experiments/	scenario configs (A-G) as YAML
visualization/	reward curves, Q-value heatmaps (offline analysis)
configs/	default hyperparameters per algorithm
utils/	seeding, logging, reproducibility helpers
model_registry.py	# registration, versioning, activation/rollback

16. Frontend Architecture

16.1 Stack

React 18, TypeScript, Vite, Tailwind CSS, Recharts for time-series/comparison charts, Zustand for simulation/WebSocket state (justified by the need for a single live-updating store shared across the spectrum, scheduler, and metrics views).

16.2 Pages

Page	Purpose & Key Elements
Dashboard	Landing overview: active simulations, recent experiment results, quick-start demo button.
Simulation Setup	Form to configure bands, emitters, receiver, and seed; validation before creation.
Live Simulation	Real-time spectrum grid, current receiver focus, running metrics; subscribes to WS spectrum_update/detection_event.
Spectrum Visualization	Waterfall-style band-occupancy view with ground-truth overlay toggle (debug mode).
Scheduler Monitor	Current policy, last N decisions, state-vector inspector.
ML Training	Launch training jobs, view reward curves via training_progress WS events.
Experiment Manager	Create/run experiments across scenarios A-G; progress tracking.
Metrics & Analytics	Pd/Pfa/latency charts per experiment, drill-down by scenario.
Baseline vs ML Comparison	Side-by-side bar/line charts; the primary judge-facing view.
Model Management	List/activate/rollback models from the registry.
Settings	API/theme configuration, simulation-only scope banner.

Each page defines: purpose, components, user actions, REST calls, WS subscriptions, loading/empty/error states (full per-page spec maintained in the design appendix to keep this document within the page limit).

16.3 Frontend Folder Structure

```
frontend/src/
  app/           # routing, providers
  pages/         # one folder per page in 16.2
  features/      # feature-sliced logic (simulation, scheduler, experiments, models)
  components/    # shared UI primitives (charts, tables, badges)
  charts/        # Recharts wrappers (SpectrumGrid, ComparisonBars, RewardCurve)
  hooks/         # useSimulation, useWebSocket, useExperiment
  services/ api/ # typed REST clients
  websocket/     # connection manager, event dispatch
  store/         # Zustand stores
  types/         # shared TS types mirroring backend schemas
  utils/ layouts/ main.tsx
frontend/public/ frontend/tests/
```

17. Security

Control	Description
Simulation-only boundary	No module, driver, or dependency provides access to physical RF hardware, SDR devices, or external radios (NFR-010); enforced via dependency allow-list review.
No real-world interception/attack	All signal generation is synthetic; no code path accepts or transmits live RF/network capture data.
API authentication (V1)	Bearer JWT with short-lived access tokens; MVP demo runs without auth on a trusted local network.
Input validation	All request DTOs validated via <code>jakarta.validation</code> (Bean Validation) annotations; reject on constraint violation with HTTP 422.
Rate limiting	Token-bucket limiter (Bucket4j) on write endpoints (e.g., 60 req/min per client) to protect the simulation worker pool.
Secrets management	Environment-variable-based secrets via <code>.env</code> / Spring <code>application.yml</code> profiles (never committed); production uses a secrets manager.
Dependency security	Automated dependency vulnerability scan in CI (e.g., OWASP Dependency-Check / <code>mvn versions:display-dependency-updates</code> for the backend, <code>npm audit</code> for the frontend, <code>pip-audit</code> for the ML microservice).
Audit logging	All model activation and configuration-change endpoints write an audit-log entry with actor and timestamp.
Data protection	No PII is collected or stored; simulation data is synthetic.

18. Testing Strategy

Layer	Scope	Example IDs
Unit — Backend	Services, repositories, reward/metric calculators.	TEST-001, TEST-002
Unit — ML	Reward function correctness, state builder, periodicity estimator.	TEST-010, TEST-011
Unit — Frontend	Component rendering, store reducers, chart data mapping.	TEST-015
Integration	API + DB + simulation worker round-trip; WS event delivery.	TEST-020, TEST-021
ML-Specific	Reward validation across edge cases, policy sanity checks, run-to-run reproducibility given fixed seed, basic model regression thresholds, distribution-shift check across scenarios A-G.	TEST-030 – TEST-034
End-to-End	Full demo workflow: create → run baseline → run ML → compare → export.	TEST-040
Performance	Load test API (NFR-001), scheduler decision latency (NFR-002), WS latency (NFR-003).	TEST-045 – TEST-047
UI	Critical flows: create simulation, run experiment, view comparison chart.	TEST-050

Target coverage: ≥ 80% for backend/ML core logic (NFR-007). CI blocks merges below threshold or on failed integration/E2E suites.

19. DevOps & Deployment

19.1 Workflow

- **Branching:** trunk-based with short-lived feature branches; PR review required before merge to `main`.
- **CI/CD:** lint → unit tests → integration tests → build Docker images → push to registry → deploy to demo environment on tag.
- **Environments:** local dev (docker-compose), CI ephemeral, demo/production-like single-host deployment.

19.2 Docker Topology

```
services:
  api:      build ./backend (Maven/Spring Boot)    ports: 8080:8080    depends_on: [db, redis, ml-service]
  ml-service: build ./ml (Python, training/inference) ports: 8500:8500    depends_on: [redis]
  frontend: build ./frontend                      ports: 5173:5173    depends_on: [api]
  db:       image: postgres:16    volumes: [pgdata:/var/lib/postgresql/data]
  redis:     image: redis:7
volumes: { pgdata: {}, model_artifacts: {} }
```

Environment configuration via `.env.example` and Spring `application-docker.yml` (DB URL, Redis URL, JWT secret placeholder — no real secrets committed). Database schema managed with Flyway migrations under `backend/src/main/resources/db/migration`; model artifacts persisted to a named Docker volume mounted by both the `api` and `ml-service` containers.

20. Observability, UX & Demo Mode

20.1 Logging & Health

Structured JSON logs with correlation IDs (`simulation_id`, `experiment_id`, `training_run_id`) on every log line (NFR-008). Health surface: `GET /health` (liveness), `GET /ready` (dependency check), `GET /metrics` (Prometheus format: request latency, WS connections, active simulations).

20.2 Standard Error Format

```
{
  "success": false,
  "error": { "code": "SIMULATION_NOT_FOUND", "message": "Simulation does not exist",
            "details": {}, "requestId": "req_0a1b" }
}
```

20.3 UX Principles

Every visualization is designed to be legible to both technical users and non-technical evaluators: spectrum activity as a color-graded grid, current scan as a highlighted band, detections as markers, and the baseline-vs-ML comparison as a simple grouped bar chart with plain-language labels (e.g., "Detected 38% faster on average").

20.4 Demo Mode (3-5 Minutes)

```
Create Simulation → Select Scenario → Run Open-Loop Baseline → Show Metrics → Run Smart Scheduler → Show Metrics → Compare Results → Explain Improvement.
```

Judges should see, in order: (1) the spectrum populate with emitter activity, (2) the fixed round-robin baseline scan pattern and its resulting Pd/Pfa/intercept-time, (3) the ML scheduler's adaptive scan pattern concentrating on active bands, (4) its resulting metrics, and (5) a final side-by-side comparison chart with the headline improvement number.

21. Complete Project Directory Structure

```
smart-rf-scheduler/
├── frontend/           # React + TS dashboard (Section 16)
├── backend/           # Spring Boot service (Section 15)
├── ml/                # RL/bandit environments, agents, training (Section 15.5)
├── simulation/        # Shared simulation-core library imported by backend + ml
├── data/              # Generated datasets, ground-truth tables (gitignored)
├── experiments/       # Scenario configs (A-G), experiment result exports
├── configs/           # simulation.yaml, receiver.yaml, scheduler.yaml, api.yaml
├── docs/              # This PRD, ADRs, API reference exports
├── tests/             # Cross-service integration/E2E tests
├── deployment/        # docker-compose.yml, Dockerfiles, k8s manifests (V1)
├── scripts/           # seed data generation, demo runner, migration helpers
├── .github/workflows/ # CI/CD pipeline definitions
├── docker-compose.yml
├── README.md
└── .env.example
```

22. Development Plan & Phases

Phase	Deliverable	Owner	Definition of Done
0	Architecture & repo setup	Architect	Repo scaffolded, CI green on empty test suite, docker-compose boots all services.
1	Simulation engine	Backend/Data Eng	All 5 emitter classes generate correct ground truth; unit tests pass (TEST-001).
2	Receiver model	Backend	Scan/tune/dwell mechanics verified against Section 9 formal definitions.
3	Baseline scanner	Backend	Round-robin scanner produces reproducible scan order for a given seed.
4	ML scheduler (bandit)	AI/ML Lead	Bandit agent trains and beats baseline Pd on Scenario A/B (Section 13).
5	Metrics engine	AI/ML + Backend	All Section 12 formulas implemented and unit-tested against hand-computed cases.
6	Backend APIs	Backend Lead	All Section 15.3 endpoints implemented, contract tests passing.
7	Frontend	Frontend Lead	All 11 pages implemented against API/WS contracts; no console errors.
8	Integration	Full team	End-to-end experiment run succeeds via UI without manual DB intervention.
9	Testing	QA Lead	Test matrix (Section 18) passes in CI; coverage ≥ 80% on core logic.
10	Demo polish	PM + Full team	3-5 minute scripted demo runs reliably twice in a row.

23. Team Ownership

Role	Responsibilities	Key Dependencies
Product Manager	Requirements, acceptance criteria, prioritization, demo script.	All leads for status/scope trade-offs.
Technical Architect	System architecture, interfaces, cross-cutting technical decisions (Section 26).	Backend, AI/ML, Frontend leads.
AI/ML Lead	Environment, reward design, agent training, evaluation, periodicity estimator.	Simulation core from Backend; metrics contract.
Backend Lead	APIs, database, simulation/receiver integration, WebSocket hub.	ML inference contract; DB schema sign-off.
Frontend Lead	Dashboard, visualizations, WS client integration.	Finalized API/WS contracts.
Data Engineer	Dataset schemas, scenario configs, experiment result exports.	Simulation and metrics schemas.
DevOps	Docker, CI/CD, environment configuration.	Finalized service boundaries.
QA Lead	Test strategy, traceability matrix, regression suite.	Stable API/ML contracts per phase.
Security Engineer	Scope-boundary review, dependency audit, secrets handling.	DevOps pipeline hooks.

24. Risks & Mitigations

Risk	Probability	Impact	Mitigation	Contingency
ML scheduler fails to beat baseline on some scenarios	Medium	High	Start with simplest algorithm (bandit); tune reward weights per scenario; validate on Scenario A/B first.	Report honest per-scenario results; keep Q-Learning as fallback comparison.
Training instability (Q-Learning/DQN)	Medium	Medium	Fixed seeds, checkpointing, small state/action space for MVP.	Freeze best checkpoint; disable further training for demo.
WebSocket performance under demo load	Low	Medium	Event coalescing (Section 15.4), load-tested before demo.	Fallback to REST polling for metrics.
Scope creep toward Deep RL too early	Medium	High	Explicit MVP/V1/V2 split (Section 7); architecture review gate before escalating algorithms.	Hard-freeze scope at Phase 4 sign-off.

Data/labeling errors in ground truth	Low	High	Unit tests per emitter class against hand-computed activity tables.	Block release of affected scenario until fixed.
Demo-day integration failure	Medium	High	Phase 10 dry-runs; scripted demo rehearsed twice before presentation.	Pre-recorded fallback video of a successful run.
Scope misunderstood as operational system	Low	High	Explicit simulation-only banner in UI and README; Section 2/16/17 scope language.	Immediate clarification in presentation materials.

25. Assumptions & Constraints

Category	Statement
Scope (constraint)	System is simulation-only; no real RF hardware, no operational deployment.
Data (assumption)	All data is synthetic and generated by the simulator; no external or classified datasets are used.
Compute (constraint)	MVP training assumed to run on a single developer machine/CPU; GPU is optional for V2 Deep RL.
Timeline (constraint)	Hackathon development window bounds MVP scope to Section 7's MVP column.
Security (constraint)	No classified or export-controlled information is used at any stage.

26. Technology Decision Records

Technology	Why	Alternative Considered	Why Not Alternative
Java + Spring Boot	Strong typing, mature Spring Data JPA/Postgres integration, first-class WebSocket support, dependency injection, and a well-understood testing story (JUnit 5 + Testcontainers) for a multi-team engineering org.	Python + FastAPI	Weaker enterprise conventions (DI, layered architecture) at this team size; retained only for the ML microservice (ADR-06) where its ML ecosystem is decisive.
PostgreSQL	Relational integrity for scan/detection event graphs; mature JSON column support for state vectors; first-class Spring Data JPA + Flyway support.	MongoDB	Weaker referential integrity for the relational entity model (Section 14.4).
React + TypeScript	Type safety across REST/WS contracts; large charting ecosystem (Recharts).	Vue	Smaller team familiarity; no functional advantage for this scope.
Gymnasium environment interface	Standard RL API; interoperable with Stable-Baselines3 for V2 escalation.	Custom-only interface	Loses compatibility with off-the-shelf DQN/PPO implementations.
Python ML microservice alongside Spring Boot (ADR-06)	Isolates the RL/bandit training and inference code — where Python's ML ecosystem is decisive — behind a small internal REST contract, while the domain/API/persistence layer stays on the strongly-typed Spring Boot stack.	Re-implement bandit/Q-Learning in Java (e.g., Tribuo, DL4J)	Smaller ecosystem, fewer maintained RL algorithm implementations, and no straightforward V2 upgrade path to DQN/PPO or Stable-Baselines3.
Contextual Bandit as MVP algorithm	Simplest algorithm demonstrating measurable improvement; minutes-scale training; interpretable per-band value estimates.	DQN as MVP	Longer training, GPU dependency, low interpretability — not justified for MVP scenario complexity.
Redis (pub/sub + queue)	Low-latency WebSocket fan-out and lightweight background job queue.	RabbitMQ/Kafka	Unnecessary operational complexity for MVP scale (NFR-004).
Docker / docker-compose	Single-command reproducible environment for dev and demo.	Bare-metal install scripts	Less reproducible across judge/reviewer machines.

27. Requirement Traceability Matrix

Req	Component	API	Database	Frontend	ML	Test
FR-001	Simulation Service	POST /simulations	simulations	Simulation Setup	—	TEST-001
FR-002	EmitterBehavior	POST /emitters	emitters	Simulation Setup	—	TEST-001
FR-005	Baseline Scanner	PUT /scheduler/config	scan_events	Scheduler Monitor	—	TEST-002
FR-006/007	Scheduler Service	PUT /scheduler/config	scheduler_decisions	Scheduler Monitor	bandit/q-learning agents	TEST-010/011
FR-008	Metrics Engine	GET /metrics/*	experiment_results	Metrics & Analytics	evaluator.py	TEST-020
FR-009	Experiment Manager	POST /experiments/{id}/run	experiments	Experiment Manager	—	TEST-040
FR-011	Model Registry	POST /models/train	models,	Model Management	model_registry.py	TEST-032

			training_runs			
FR-012/013	WebSocket Hub	/ws/v1/simulations/{id}	—	Live Simulation	—	TEST-021
FR-015	Periodicity Estimator	GET /scheduler/decision	scheduler_decisions	Scheduler Monitor	periodicity_estimator.py	TEST-011

28. Definition of Done, Future Scope & Appendix

28.1 Project-Level Definition of Done

1. A simulated RF environment can be created with configurable bands, duration, and seed.
2. All five emitter behavior classes can be simulated with correct ground truth.
3. The receiver model correctly enforces instantaneous bandwidth, dwell, and tuning-delay constraints.
4. The open-loop baseline scanner runs deterministically for a given seed.
5. The ML scheduler produces scan decisions that adapt to observed activity.
6. Pd, Pfa, and average intercept time are computed correctly against hand-verified test cases.
7. Baseline vs. ML results can be compared for the same scenario/seed.
8. Periodic-emitter prediction measurably improves detection latency on Scenario B.
9. All Section 15.3 APIs function end-to-end from the dashboard.
10. The system runs entirely without real RF hardware, confirmed via dependency audit (Section 17).
11. The 3-5 minute demo runs reliably twice consecutively.

28.2 Future Scope Summary

V1: Q-Learning scheduler option, full 7-scenario suite, periodicity feeding into state, model registry with rollback, authentication. **V2:** Deep RL (DQN/PPO), neural contextual bandit, multi-receiver scenarios, transfer learning across scenario families, automated hyperparameter search, adversarial frequency-agile stress testing.

28.3 Appendix — Glossary

Pd	Probability of Detection	Pfa	Probability of False Alarm
Dwell time	Minimum time the receiver observes a tuned band	SNR	Signal-to-Noise Ratio
DQN	Deep Q-Network	PPO	Proximal Policy Optimization

End of document. This specification defines a simulation-only research prototype; no section authorizes real-world RF interception, jamming, electronic attack, or weapon-control capability.